

Programming Quests

Bryan C. Souza 

Abstract

Exercises for becoming an awesome programmer

1. PROGRAMMING QUESTS

Exercises for becoming an awesome programmer

This is a living document with a collection of programming exercises, sometimes also referred as quests. The goal of these quests is to provide interesting opportunities for novice programmers to practice their skills, while learning other topics and (hopefully) having fun at the same time. The exercises are designed in Python, but feel free to try to solve them with other programming language.

Note

Each exercise clarifies the required skills needed at the beginning, together with the topic. For example, **A Study with Strings** requires basic knowledge of str, loops and if statements.

1.a. About the author

I'm a [lecturer at Radboud University](#), interested in making programming fun and inclusive. Except when stated otherwise, the exercises you can find on the website were designed by me.

Because I care deeply about bias, the environment and ethical issues around the current generative AI models, no LLM was used in any way while building these resources and this website. This means that every quest was deeply thought of and designed to fit a specific level and interest that I find relevant.

2. LINEAR ALGEBRA

This chapter will help you get consolidate your knowledge of linear algebra. To solve the exercises in this section you will need to have some basic understanding of:

- numpy arrays
- loops and if statements
- functions

2.a. What is Linear Algebra

“The heart of linear algebra is in two operations - both with vectors. We add vectors to get $v + w$. We multiply them by numbers c and d to get cv and dw . Combining those two operations (adding cv to dw) gives the linear combination $cv + dw$ ”

— Strang (2009)[p.1]

An arrow in a plane is often used as a simple representation of a vector. Using two orthogonal axis x and y (i.e., angled with 90 degrees), one can describe such vector as a pair of coordinates (a, b) , with a representing the coordinate in x , and b representing the coordinate in y . The vector can then be drawn from the origin - the point $(0,0)$ - to the point (a,b) .

Such a vector is said to exist in a two-dimensional plane, because it has two coordinates (a and b). However, once one goes beyond the arrow and plane analogy, nothing limits the amount of dimensions of a vector: $(1,4,5)$ is a three-dimensional vector, $(5,2,-6,2)$ is a four-dimensional vector, and so on.

When translating vector coordinates to programming, one can think of different ways to store them. For instance, tuples and lists satisfy the need of storing multiple elements in an ordered fashion. However, arrays are specially suited for that because they are built in a way that optimizes the two operations discussed above, and the operations that derive from them. Thus, in Python, linear combinations are often computed using arrays, specifically with the Numpy module (imported as `np`). Because of that, from here on whenever we use the terms array, the reader should assume we refer to numpy arrays.

Note also that there is a key distinction when talking about arrays in the context of a computer program and vectors in the context of algebra. For instance, we might use the (numpy) array `a = np.array([1,3,5])` to represent the three-dimensional vector $(1,3,5)$. However, the variable `a`, in this case, is said to be a (numpy) array of one dimension. This is so, because the indices of that array can be arranged in a line.

This distinction between arrays in the context of a computer program, and vectors and matrix in the context of linear algebra also extends to the concept of lenght of an array or vector. For instance, the length of the two-dimensional (2D) vector a with coordinates $(3,4)$ can be calculated using Pythagoras' theorem to find the hipotenusa of the triangle formed by the vertices $(0,0)$, $(3,0)$, $(3,4)$:

$$h = \sqrt{a^2 + b^2}$$

with $a = 3$ and $b = 4$, we have that the length (also known as the module of the vector) of that vector is $h = \sqrt{3^2 + 4^2} = 5$.

On the other hand, the length of the array `a = np.array(3,4)` can be retrieved by the calling function `len(a)`, and in this case is 2, which is the number of elements in the array. (The length of the array is actually equivalent to the dimension of the vector)

```
a = np.array(3,4)
len(a)
```

Because of that, in the rest of this section we make an explicit distinction between the words “vector” and “array”. With vectors (and matrices) being used in the context of linear algebra as explained above, and arrays in the context of programming.

2.a.i. Exercise 1:

The code below defines two vectors (represented by the arrays `a` and `b`) and plots them in a 2D plane. Write the definition for the function `plot_vector()`. The function should receive an array containing a 2D vector as a parameter (the length of the array should be 2, then!) and a plot axis. The function should plot the vector stored in the array in the respective figure axis.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
a = np.array(3,4)
b = np.array(-5,6)
```

```
fig, ax = plt.subplots()
plot_vector(ax, a)
plot_vector(ax, b)
plt.xlim((-16,16))
plt.ylim((-16,16))
plt.show()
```

`image("files/33a14d2cbe8dc978308f5e892eb73ad3.png", width: 90%)`

```
def plot_vector(arr, ax):
    # Your code goes here:

    return
```

2.a.ii. Exercise 2:

Write a function `linear_combination()` that receive two 1-D arrays representing two N-dimensional vectors u and v , and a third 1-D array w with two elements c and d . Your function should return the linear combination $cu + dv$.

```
def linear_combination(u,v,w):
    # Your code goes here

    return
```

2.a.iii. *Exercise 3:*

Write the function `vector_length()` that receives an array `arr` as a parameter and returns the length/module of the vector stored in `arr`. Your function should work for vectors of any dimension greater or equal than 2.

```
def vector_length(a):
    # Your code goes here:

    return
```

2.b. *Unit vectors*

The coordinates of a vector carry essentially two information: a direction in the vector space (the set of axis in which the vector exists), and a magnitude (the length/norm of that vector). Sometimes, one may need specifically the direction the vector represents, but not its magnitude. In such cases, it is usefull to compute what we call a unit vector u from a vector v . u is a vector in the same direction of the original vector v , but with unitary length ($|u| = 1$).

2.b.i. *Exercise 4:*

Write a function `unit_vector()` that receive an arrays representing N-dimensional vector v and returns the unit vector of v .

```
def unit_vector(arr1):
    # Your code goes here:

    return
```

2.c. *Dot product*

The dot product is an operation between two vectors of the same dimension and results in a number or scalar (often this operation is also called the scalar product, or the inner product between the two vectors).

One way of computing the dot product is using the individual coordinates of each vector. For instance, the dot product between two 2D arrays $v = (v_1, v_2)$ and $w = (w_1, w_2)$ can be computed using the following formula:

$$v \cdot w = v_1 w_1 + v_2 w_2$$

This can be generalized to N dimensions as:

$$v \cdot w = \sum_{i=1}^n v_i w_i$$

2.c.i. Exercise 5:

Write a function `dot_product()` that receive two arrays representing N-dimensional vectors and returns the dot product between the arrays.

```
def dot_product(arr1, arr2):
    # Your code goes here:

    return
```

There is another way to compute the dot product between two vectors, by using the geometrical interpretation of what is this operation. Given the two vectors v and w , the dot product can also be computed using the following formula:

$$v \cdot w = |v| |w| \cos(\theta_{vw}) \quad (1)$$

Where $| |$ is the module operation, which returns the length of a vector, and θ is the angle between the two vectors.

This new formula allow us to understand the inner product in a geometrical way because the quantity $|w| \cos(\theta_{vw})$ is the “projection” of the vector w in the axis of v (analogously, $|v| \cos(\theta_{vw})$ is the projection of the vector v in the axis of w). To visualize this quantity, one can see the “shadow” that w projects into v as depicted in red in the figure below, where $w = (3, 4)$ and $v = (8, 2)$.

image("files/73c64d2c3fc65d843f97255093e56cba.png", width: 90%)

Hence, following the formula in (1), the dot product between v and w is *the length of v* (or $|v|$) multiplied by *the length of the projection of w in v* (red arrow).

2.c.ii. Exercise 6:

Write a function `angle()` that receive two arrays representing N-dimensional vectors and returns the angle between them. Use the functions implemented in the earlier exercises and the formula in (1).

```
def angle(a,b):
    # Your code goes here

    return
```

3. LINEAR ALGEBRA

This chapter will help you get consolidate your knowledge of linear algebra. To solve the exercises in this section you will need to have some basic understanding of:

- numpy arrays
- loops and if statements
- functions

3.a. Matrices

In the previous section, we saw how to do linear combination of vectors v and w as described in the equation below.

$$cv + dw = b$$

with c and d as scalars and b as a resulting vector.

Notice that to do that, we had to operate on each vector before combine them through addition. However, one can see that operation from a slightly different perspective if we combine the two vectors beforehand, in a kind of vector of vectors.

Let's see this example using the vectors $v = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ and $w = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ (Notice that we are representing the vectors as column-vectors in order to facilitate the visualization of the equations).

The linear combination then becomes:

$$c \begin{bmatrix} 1 \\ -1 \end{bmatrix} + d \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} c \\ d-c \end{bmatrix}$$

If we stack the two vectors inside another vector we get:

$$A = \begin{bmatrix} v & w \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} & \begin{bmatrix} 0 \\ 1 \end{bmatrix} \end{bmatrix}$$

Where A is a matrix. Note that matrices representation are often a simplified without the inner brackets:

$$A = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Now we can rewrite the previous linear combination using a matrix multiplication of A and a column-vector x with the values of c and d .

$$x = \begin{bmatrix} c \\ d \end{bmatrix};$$

$$c \begin{bmatrix} 1 \\ -1 \end{bmatrix} + d \begin{bmatrix} 0 \\ 1 \end{bmatrix} = Ax = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} c \\ d \end{bmatrix} = \begin{bmatrix} c \\ -c+d \end{bmatrix}$$

This is useful because we are now shifting from the previous perspective, in which the numbers (c and d) were multiplying the vectors. Now, the matrix (formed by the vectors) is multiplying the numbers (now in the vector x). The linear combination equation can now be rewritten as:

$$Ax = b$$

In some problems, we will be interested in finding the resulting vector b for a given x . But in others, we will be interested in finding which x gives a particular b .

You will also see later that some matrices have specific properties that are independent of which numbers are in x . For example, the matrix A below is a rotation matrix that will rotate any 2D vector by 90 degrees.

$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

Matrix multiplication and linear systems

The matrix multiplication $Ax = b$ represents a system of linear equations when x is unknown. For example, given

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \text{ and } b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

We have the following system of equations:

$$Ax = b \rightarrow \begin{bmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

3.b. Storing matrices in arrays

Since matrices are basically ‘vectors of vectors’, it makes sense that we use arrays of arrays to store them in a computer. We can define the matrix A from the session above using something like:

```
import numpy as np

A = np.array([[0, -1], [1, 0]])
```

Notice that now, `m.shape` is `(2, 2)`, and hence, `m` is a 2D array. One could choose to use more (array) dimensions to store a 2D matrix. However, those extra dimensions would only have one value. For example, the code below defines `m` as a 4D array (since the shape is now `(1, 1, 2, 2)`), even though it represents a 2D matrix. The two extra first dimensions have only a single value.

```
A = np.array([[[[0, -1], [1, 0]]]])
print(A.shape)
```

A possible reason to do that might be that one needs to store many different matrices (let’s say, coming from different experiments). This could be done in one of those extra dimensions, without compromising the ability to operate across dimensions of those matrices. In that case, those extra dimensions would not have only a single value.

3.c. Matrix multiplication

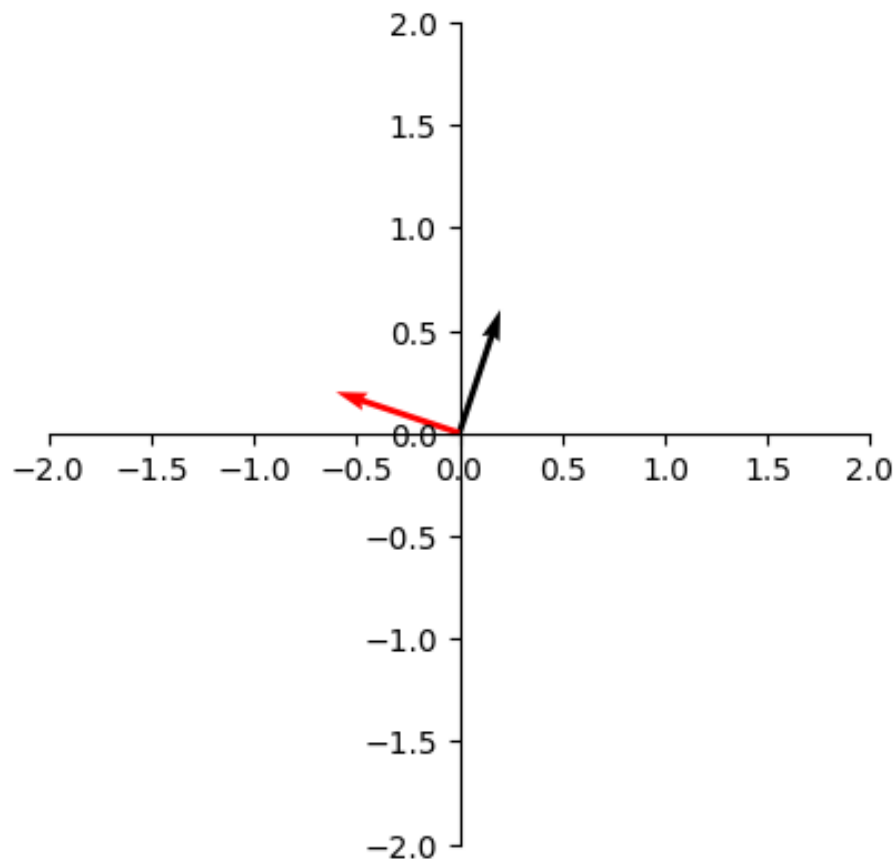
The matrix multiplication operation definide above can then be done using the function `np.matmul()` or the `@` operator.

```
import numpy as np

A = np.array([[0, -1], [1, 0]])
x = np.array([0.2, 0.6])

# with matmul
b = np.matmul(A,x)
# with @
b = A@x
```

See the original and resulting vectors in the plot below.



3.c.i. Exercise 1:

Write your own function to implement the multiplication of a vector by a matrix. You will often use the implementation of numpy, but it is useful to implement yourself at least once.

Play around with different vectors, seeing how they all rotate 90 degrees, without any other change when multiplied by this matrix.

Multiplication of matrices

One can also multiply a matrix by another matrix. This is similar to doing multiple matrix and vector multiplications and stacking the resulting vectors in a matrix as well. You can try to make your function also deal with this case as a challenge.

Notice that the size of the resulting matrix is always defined by the size of the multiplying matrices. For instance, a n by m matrix A , multiplied by a p by q matrix B will result in the n by q matrix C . For the matrix multiplication to be possible, m and p must also be equal.

Hence, the following multiplication is possible.

$$A_{2 \times 3} * B_{3 \times 5} = C_{2 \times 5}$$

But the opposite operation ($B * A$) is not valid.

```
def matrix_multiplication(A, x):
    # Your code goes here.

    return b
```

3.d. Transposition, trace and determinants of a matrix

Given a matrix A , populated by elements a_{ij} , with i representing the rows of the matrix and j are its columns, the transpose of A is the matrix A^T . The elements a_{ij}^t of A^T are such that an element in row i and column j in A^T correspond to the element in row j and column i in A . In other words:

$$a_{ij}^t = a_{ji}$$

You can easily compute the transpose of a matrix m in numpy using `np.transpose(m)` or `m.T`.

3.d.i. Exercise 2:

Write your own function that computes the traspose of a matrix.

Inner product and outer product

Now that we have defined the transpose of a matrix, we can do more easily do a matrix multiplication with two vectors. Given the 3-by-1 column vectors v and w , it's now possible to arrange them so that a matrix multiplication is possible.

The first way of doing that is by computing:

$$v^T w \tag{2}$$

The resulting operation is a 1-by-1 (thus, a scalar). If you unpack the multiplications done in (1), you will see that it's the exact same as the formula of the dot product in (1).

One can also do the opposite operation, transposing the second vector to match the dimensions. In that case, the result would be a 3-by-3 matrix.

$$vw^T \quad (3)$$

This operation is known as the **outer product** (in contrast with the inner product). The inner product of two vectors is actually the trace of their outer product (see the next exercise for the trace definition).

```
def matrix_transposition(A):
    # Your code goes here.

    return
```

There are some other quantities associated to a matrix that are a single value, rather than another vector or matrix (like the transposition or outer product). One of these quantities is the trace. The trace of any square matrix can be defined as the sum of all the elements in the diagonal.

For every element a_{ij} of a matrix A , the trace can be defined as

$$Tr = \sum_{i=j} a_{ij}$$

where i represents the row, and j the columns of A

3.d.ii. *Exercise 3:*

Write the function `trace` that computes the trace of a 2D square matrix.

```
def trace(m):

    return
```

Another important quantity that can be defined from a matrix A is the determinant of the matrix. This quantity is related to, among other things, the linear system described by the matrix A . The linear system has a solution, if and only if the determinant of the matrix is different than 0. If the determinant of a matrix is 0, this matrix is said to be singular.

For a 2-by-2 square matrix $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$, the determinant $\det(A)$ (or sometimes $|A|$) is defined by the formula:

$$\det(A) = ad - bc$$

Some interesting property of determinants are that:

$$\det(AB) = \det(A)\det(B)$$

And if A is a triangular matrix, $\det(A)$ is the product of its diagonal.

3.d.iii. *Exercise 4:*

Write the function `determinant` that computes the determinant of any n-by-n triangular matrix, or of any 2-by-2 square matrix.

```
def determinant(m):
```

```
    return
```

3.e. *Identity matrix and inverse matrix*

Like addition and multiplication, the operation of matrix multiplication also has a “neutral” element. In the case of addition and multiplication, one can find the neutral element by finding the value x for which the following equations hold for any value of a :

$$a + x = a; \text{ and } ax = a$$

It is easy to see that $x = 0$ in the first case, and $x = 1$ in the second. For matrix multiplication, the neutral element I should satisfy the equation:

$$AI = A$$

I , also known as the identity matrix, has all its elements in the main diagonal (the elements a_{ij} where $i = j$) as 1, and all other elements as 0. For a given matrix A , the size of the corresponding identity matrix depends on the size of A . For instance, for a 3-by-3 matrix, I_3 is defined as:

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The identity matrix also help defining another important matrix, the inverse matrix.

Just as -3 is the inverse of 3 regarding addition ($3 - 3 = 0$). And 0.5 is the inverse of 2 regarding multiplication ($0.5 \cdot 2 = 1$). We can find the inverse matrix of A , represented as A^{-1} , such that:

$$AA^{-1} = A^{-1}A = I$$

Note that the determinant can also help defining if a matrix can be inverted. This is only the case when the determinant is different than 0.

3.e.i. *Exercise 5:*

Write a function `is_inverse()` that receive two arrays representing n-by-n matrices, and returns a True if those matrices are the inverse of each other, and False otherwise.

```
def is_inverse(m1, m2):
```

```
    return True
```

Challenge

The inverse of a matrix can be found by applying the [Gauss-Jordan elimination process](#).

Read about that method, and implement a function `invert()`, that uses this technique to find the inverse of a matrix.

4. NEURO SIGNAL PROCESSING

This chapter will help you get develop your numpy array skills in Python. To solve the following exercises you need to have some basic understanding of:

- arrays and indexing
- loops and if statements
- basic plotting (you might also learn as you go)

4.a. *Simulating and plotting signals*

The code below uses numpy arrays to simulate an oscillatory signal, similar to an EEG or LFP signal. It first start by creating an array `time` with time stamps. Those are the times in which the simulated electrode sampled the voltage activity of the brain. Because most recording systems have a fixed sampling time, we can just create an array `time` with equally spaced times, denoted by the variable `dt` (referring to Δt , the difference between two consecutive time samples t_i and t_{i+1}). In this case, time starts at time 0 and stops at time 100 (we will assume that time and `dt` are in seconds).

EEG and LFP signals

In neuroscience there are multiple ways to measure the activity of the brain. One of the most common ones is by using voltage as an indicator of neuronal activity (neurons change their internal voltage whenever they fire an action potential). In humans, it's common to put an electrode in contact with the head skin to measure the voltage changes occurring within the brain. That signal, the electroencefalogram signal (or simply EEG), is usually quite noisy, and it's hard to find its exact source, since the electrode is so far from a particular source. However, since we are very good in measuring voltage, it has a good temporal resolution.

Whenever it is possible to place an electrode inside the brain (e.g., if you are already going to do an invasive procedure anyways or if you are working with animal models) it's possible to get a slightly more accurate signal. From that signal, one can look both into the activity of individual neurons very close to the tip of the electrode - known as spiking activity - as into the general activity of neurons in the region - known as the local field potential (LFP). The later one, is very similar to the EEG signal, except that in this case you can more easily pinpoint where the signal comes from based on the coordinates of the implant.

A good analogy to understand these electrode recordings and the type of signal they convey is to think of someone calling you from a party. Maybe you can hear what your friend and a few other people that are very close to the phone are talking. That is the spiking activity. But you can also hear the general chattering in the background (without recognizing any word, but knowing it comes from a particular room). That's the local field potential. At last, it may be that the only thing you can hear is a bit of the chatter covered by the loud music playing, which can be heard from everywhere in the party. That is the EEG signal.

In this analogy, every person in the party is a neuron, your phone is an electrode inside the brain or in the scalp, and the music playing is what we call “noise”, a signal that does not carry any information about the neurons. Of course this analogy is not perfect, but it helps conveying the idea of what those signals are.

```
import numpy as np
import matplotlib.pyplot as plt

dt = 0.001
time = np.arange(0, 100, dt)
```

Note that the Δt can also be defined in terms of the sampling rate (*srate*) of the recording system simulated here by the following equation:

$$\Delta t = 1/\textit{srate}$$

After that, we can use our time array to create two pure oscillations: one in 8 Hz and another in 25 Hertz. This can be done by using a cosine (or sine) function with an angle that varies over time

$$s(t) = \cos(\theta(t))$$

The rate in which θ changes defines the frequency of our oscillation. Because the cosine function receives a frequency in radians (ω), if we want to define our oscillation using a frequency in Hertz (f) we need to use the following equivalence:

$$\omega = 2\pi f$$

Hence, our oscillation becomes

$$s(t) = \cos(2\pi f(t))$$

Well, the last step is just to consider the function $f(t) = f_{const}t$, in which f_{const} a constant frequency denoted by the variables *f1* and *f2*, and t is the time, denoted by the variable *time*.

```
import numpy as np
import matplotlib.pyplot as plt

dt = 0.001
time = np.arange(0, 100, dt)

f1 = 8
f2 = 25
sig1 = 0.5*np.cos(time*2*np.pi*8)
sig2 = 0.2*np.cos(time*2*np.pi*25)
```

Note that each signal is multiplied entirely by a scalar (0.5 or 0.1). This is just to scale each signal to a different size. In real life, the size of signals depend on factors like the resistance of the electrode, the amplification of the recording system and so on. But

there is a natural tendency of faster frequencies to have less amplitude in biological systems.

At last, we simulate some noise by creating a random array of the same size of our signal and adding it to it before plotting.

```
import numpy as np
import matplotlib.pyplot as plt

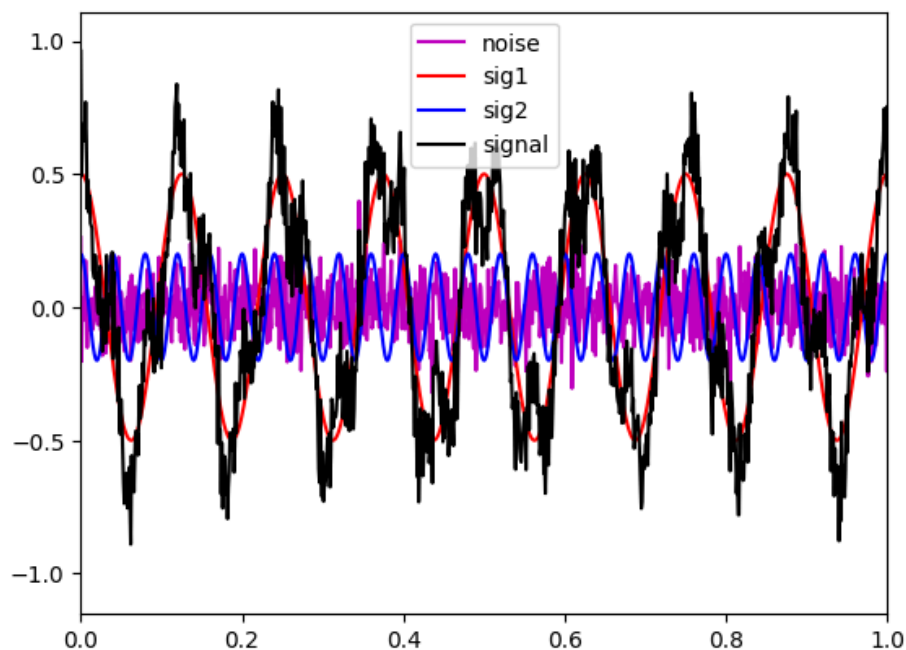
dt = 0.001
time = np.arange(0,100,dt)

f1 = 8
f2 = 25
sig1 = 0.5*np.cos(time*2*np.pi*8)
sig2 = 0.2*np.cos(time*2*np.pi*25)

noise = 0.1*np.random.randn(time.shape[0])
signal = sig1+sig2+noise
```

We can now plot each individual oscillation, the noise, and the sum of all of them, comprising our simulated signal.

```
plt.plot(time, noise, 'm')
plt.plot(time, sig1, 'r')
plt.plot(time, sig2, 'b')
plt.plot(time, signal, 'k')
plt.xlim((0,1))
plt.legend(['noise', 'sig1', 'sig2', 'signal'])
plt.show()
```



That is a very simple example of a simulated signal. Now imagine that the oscillations depicted above in black is the signal that arrives from one of your EEG or LFP recordings. You can clearly see that there are oscillations happening there (the 8 Hz is quite visible!), but it's hard to disentangle anything else. What can we do then, to understand what's going on in this signal?

One possibility is to filter the signal. Filtering is a procedure that allow us to remove or isolate certain frequencies in our signal. You can understand a bit more about filtering by studying [convolution](#) and the [Fourier transform](#), but for now, let's use the following function below to filter our data.

```
import scipy.signal as sig
def filter(signal, f_low, f_high, srate):
    b, a = sig.butter(
        N=4,
        Wn=(f_low, f_high),
        btype="bandpass",
        fs=srate,
    )

    filtered = sig.filtfilt(b, a, signal)

    return filtered
```

The function will receive a signal (`signal`), a lower bound frequency `f_low` and an upper bound frequency `f_high`, and the sampling rate (`srate`), and return a filtered signal containing only the frequencies between `f_low` and `f_high`.

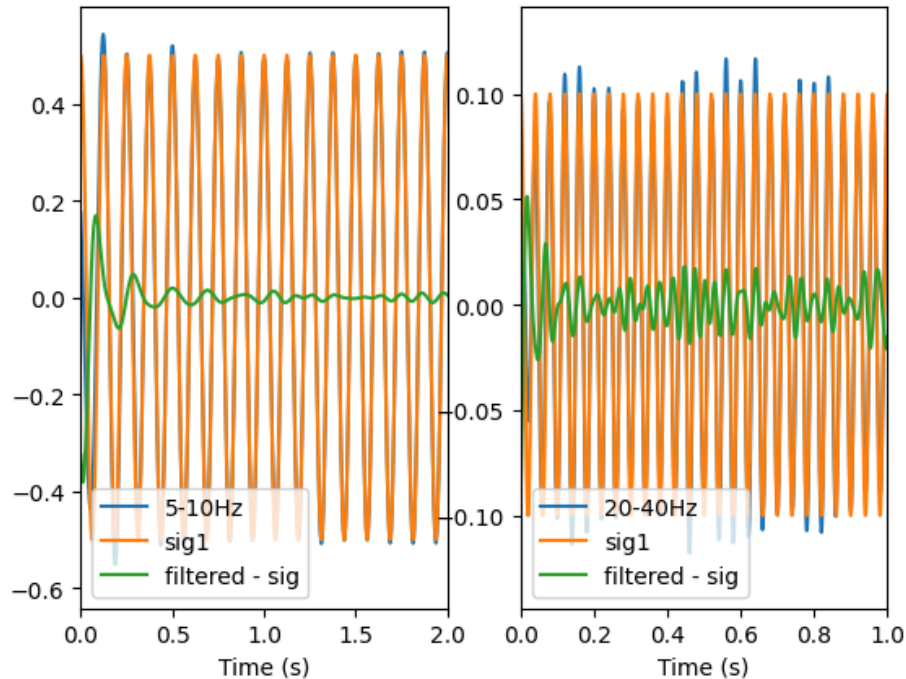
4.a.i. *Exercise 1::*

Use the filter function above to filter two components present in `signal`: one from 5 to 10 Hz, and another from 20 to 40.

Plot the two components in a new plot. Also plot the difference between the original `sig1` and `sig2` with their respective filtered components.

Your code goes here.

<matplotlib.legend.Legend at 0x22663944d90>



4.a.ii. Exercise 2::

Use the functions `np.mean()` and `np.std()` to calculate the average (μ) of and standard deviation (σ) of the synthetic signal generated (signal). Use that to define a threshold of $\mu + 1.5 * \sigma$

Plot the signal (in black) and the threshold (as a dashed line). In addition to that, replot all the signal traces that are above the threshold (as disconnected dots in red)

Limit your plot to the first 1 second.

Your code goes here.

4.b. Part 2

The functions `np.where` can be very useful to find indices of an array that fulfill certain conditions. For example, look at the code below:

```
import numpy as np

arr = np.array([1,2,3,5,5,7,10])
indices = np.where(arr==5)
```

The variable `indices` is the tuple (`array([3, 4]),`) in which the first element is an array with the indices of `arr` in which the condition `arr==5` is True.

This means we can use indices to access all the elements of `arr` that fulfill the condition:

```
print(arr[indices[0]])
```

One might ask, how is this different than `arr[arr==5]`? It is true that they result in the same operation. However, sometimes one needs to do operations in the indices themselves. For example, imagine that you have a signal and want to get periods in which the signal is higher than a threshold for 3 consecutive points.

You could do something like `signal[signal>threshold]` to get the values above the threshold, but that does not guarantee that you will have consecutive activation. Instead, if you use `np.where`, you can find the indices where the condition is True.

```
idx = np.where(signal>threshold)[0]
```

Imagine that the resulting vector is something like:

By looking at it, we can see four periods in which the signal was above the threshold: 2-3, 10-12, 21-25 and 30 (also shown in the spacing used). Only two of them fulfill our criteria of three consecutive points (10-12 and 21-25). Let's see how we can get this information from the code:

if we do the operation `diff = idx[1:] - idx[:-1]` we are basically creating a new array in which each value is the difference between two consecutive values of our `idx` array (e.g., `diff[0]` is `idx[1]-idx[0]`; `diff[1]` is `idx[2]-idx[1]`, and so on). Note that this new array `diff` has one less element than `idx` (because there is no previous value to subtract from the first element). In our example, `diff` would be:

```
diff = idx[1:] - idx[:-1]
print(diff)
```

Note that whenever `diff` is different than 1, this represents a two points that are not consecutive. We could then use `np.where` to find all the transition points between two activation periods by doing something like:

```
transitions = np.where(diff>1)[0]
print(transitions)
```

Now we would have transitions as the array: `[1,4,9]`

Which will always be the length of the amount of contiguous periods that we have minus one (4-1 in this case). We now know where in `idx` is the beginning and end of a contiguous period:

- The first period goes from `idx[0]` to `idx[transitions[0]]` (2-3)
- The second period goes from `idx[transitions[0]+1]` to `idx[transitions[1]]` (10-12)
- The third period goes from `idx[transitions[1]+1]` to `idx[transitions[2]]` (21,25)
- We can build a last period from `idx[transitions[2]+1]` until the end of the array: `idx[transitions[2]+1]` to `idx[-1]`

So we can build an array with the starting indices of the contiguous periods:

```
start = np.concatenate([[idx[0]], idx[transitions+1]])
print(start)
```

And similarly the ending indices:

```
end = np.concat([idx[transitions], [idx[-1]]])
print(end)
```

Note that we added `idx[0]` and `idx[-1]` because even though these points are part of a contiguous period, they are not captured by the transitions, since there is nothing before or after them.

Note that `start[0]` corresponds to the start of the first threshold activation. And `end[0]` corresponds to the end of such activation. We can simply define:

```
duration = end - start + 1
print(duration)
```

to get the duration of each activation.

And from there, we can filter the activations by size:

```
start = start[duration>=3]
end = end[duration>=3]

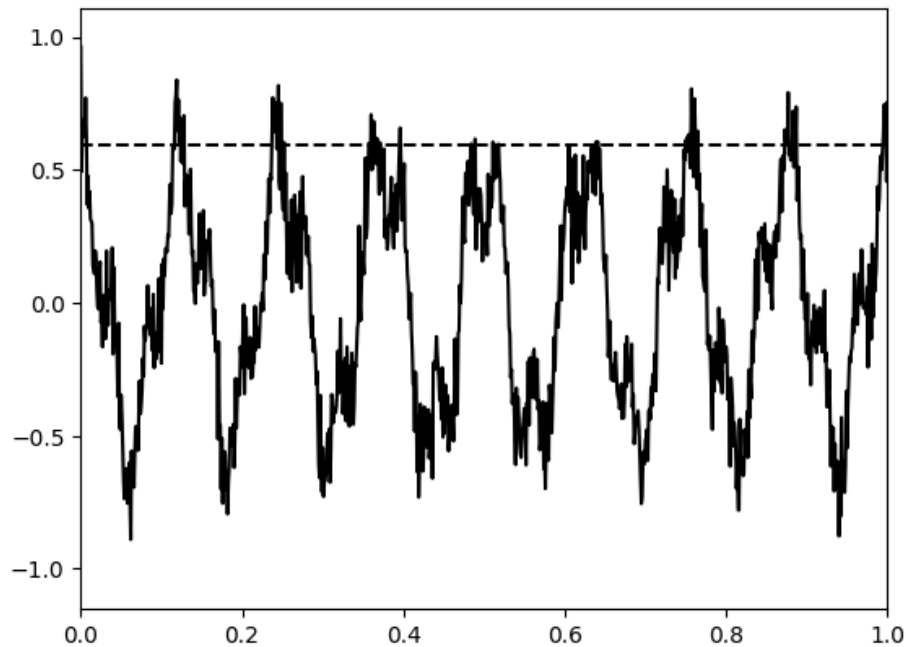
print(start)
print(end)
---
```

4.b.i. Exercise 3::

Write a script that finds all the periods in which the signal created in the previous section is above the threshold of *1 standard deviation over the mean* of the signal. Only select threshold activations *higher than 0.01 seconds* (remember you have the array time!).

At the end, plot the threshold (dashed line), the signal (in black) and periods of activation (in red).

```
# Your code goes here
```



4.b.ii. Exercise 4::

As we saw above, the function `np.diff()` can be used to compute the difference between consecutive values in an array. So, for an array `arr`, `np.diff(arr)` is equivalent to: `arr[1:] - arr[0:]`. In a way, this quantity is related to the discrete derivative, and hence name `diff`, from differential.

```
print(np.diff([1, 3, 4, 10]))
[2 1 6]
```

As you might have learned in Calculus, the differential can be used to find local minima and local maxima in an array (inflection points). Of course, because in a computer values are stored in a discrete way, an inflection point of an array `arr` can be found not by finding where `np.diff(arr) == 0`, but where `np.diff(arr)` changes from positive to negative.

Use the `np.diff` function to find all the peaks of the 8 Hz component found in the previous filtering exercise.

Sign of a signal

Since to find transition points one only need to know the sign of the value in a specific position, one could simplify the derivative of the signal by only looking if a the values are positive or negative. How could you do that using array operations?

Tip

You can do that by hand. But numpy also have an implementation for that.
Check the function `np.sign()`.

Your code goes here

5. INTRODUCTION TO CRYPTOGRAPHY

This chapter will help you get consolidate your knowledge of functions and indexing (in strings). To solve the following exercises you need to have some basic understanding of:

- str indexing
- loops and if statements
- functions

By following these challenges you will also learn about concepts related to cryptography (e.g., frequency analysis, Krawinski method), and the history of a few Arab scientists whose contributions are related to STEM fields.

5.a. *The substitution cipher*

There are many contexts in which one would not want that a piece of information is available to others. In the modern world examples are the password of an email, the code of a locker or even a phone number. In fact, the need to protect information has long existed in humanity, and have decided the fate of history multiple times.

One of the first ways developed of securing information from unwanted eyes was to use a substitution cipher. In that method, the sender of a message tries to hide it's meaning by replacing each letter in the message by another predetermined letters. That process is called encryption and in that context, the original message is often referred to as the plain text, while the final message is referred as ciphertext.

In order to recover the message, the receiver of the encrypted message only needs to know which letter in the ciphertext corresponds to each letter in the plaintext (also know as the cipher key).

(You can read more about substitution cipher [here!](#))

Your first task is to write the function `substitution_cipher()` that receives three string parameters: `plaintext`, `alphabet`, and `key`.

The function should perform a substitution encryption in the plaintext by replacing each letter in alphabet by the equivalent letter in key.

Ex.:

```
key = "kzhjcfdlbtvwxmnaioyepgsuq"
alphabet = "abcdefghijklmnopqrstuvwxyz"

cipher_text = substitution_cipher('my encrypted message', alphabet, key)
```

If your program is correct, the variable `cipher_text` should be:

XU CMHRUAYCJ XCOOKDC

Note that in this and other examples decoded texts are represented in lower case while encrypted text is in UPPER case. In addition to that, notice that each letter should have a unique corresponding key, otherwise the message might not be able to be retrieved later.

```
def substitution_cipher(plain_text, alphabet, key):
    # Your code goes here:
```

```
    return
```

Similar to the function above, create also the function `decode_substitution()` to decode the cipher_text based on the key used for encryption.

In the example above, that would result in something like:

```
decoded_text = decode_substitution(cipher_text, alphabet, key)
print(decoded_text)
```

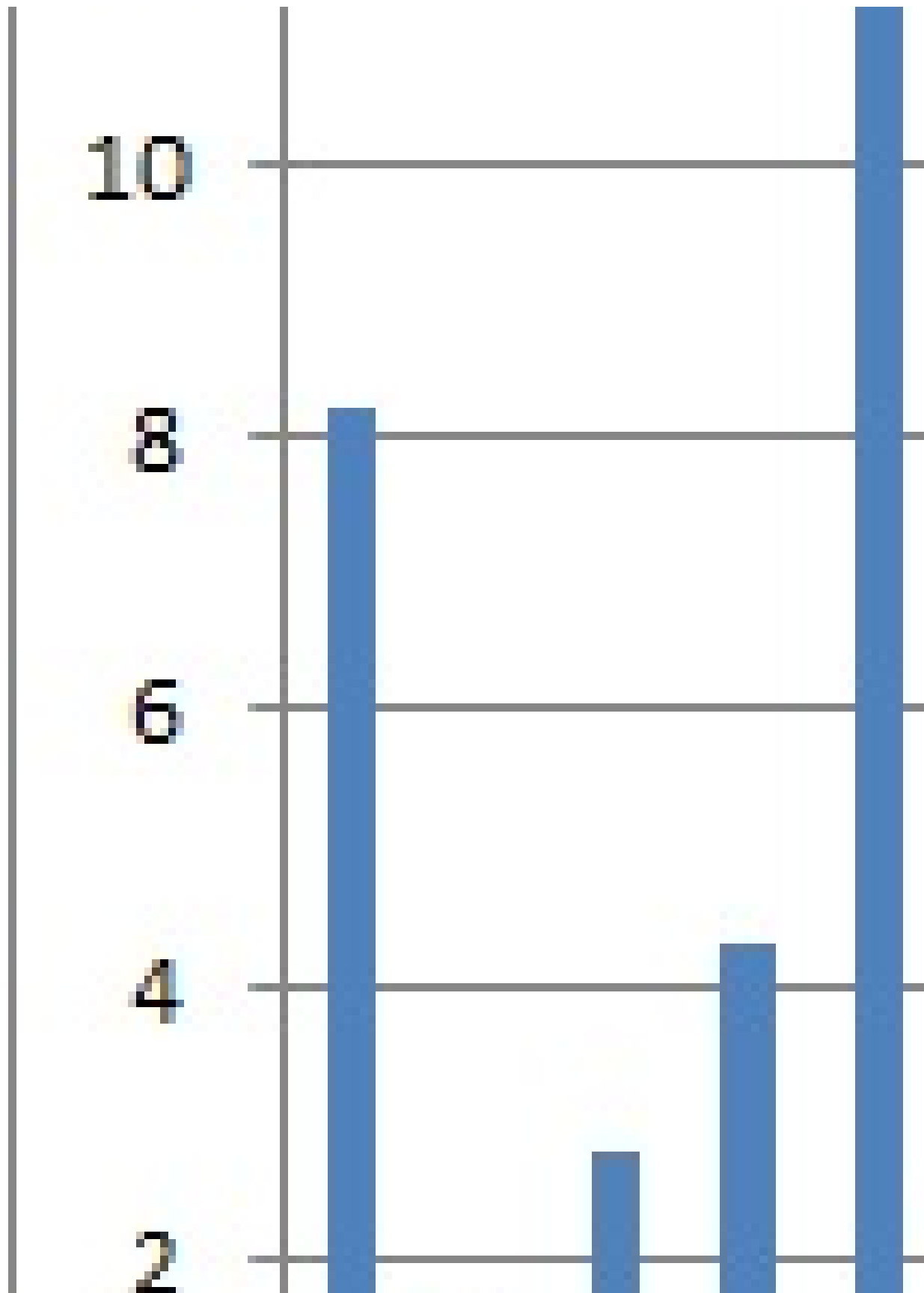
my encrypted message

```
def decode_substitution(cipher_text, alphabet, key):
    # Your code goes here:
```

```
    return
```

5.b. Frequency analysis

It may seem that this encryption method is secure if you don't know the key to decrypt. However, there are simple ways to find out the key based on statistics of the message. For instance, we can use frequency analysis to uncover what is the most probable correspondence between letters.



Frequency of each letter of the English alphabet. Source: [Crypto corner](#)

This method was first introduced by the important arab mathematician [Al-Kindi](#), the father of the cryptography field and it consists of estimating the how often each letter appear in the message, and comparing it to how often each letter appear in the language in which the message is written. Of course this assumes that one knows the original language of the message!

طليبا حيا بعد والدرى على البحر والبر
 والركب على السرى ملأ العباد من
 مع وحد ولعل من الحار والبارد
 ر بالصد والطار الفسح

پروایست علم و معجزه و تالیف

وحيث انه وجد
 في اسفل العنبر
 في كل واحد من الحبل الاسفل مارمر

The first page of al-Kindi's manuscript "On Deciphering Cryptographic Messages", containing the oldest known description of cryptanalysis by frequency analysis. Source: Public Domain

Now that you know how to implement a substitution cipher, let's try to create some functions to help us break the encryption.

Write the function `count_letters()` that receives a str message and returns a list with how many times each letter of the alphabet appears in message. (You can return a list with the counts in alphabetical order, or a dictionary)

Ex:

```
count = count_letters("'on deciphering cryptographic messages' contains
the first known description of frequency analysis!")
print(count)
```

```
[5, 0, 6, 2, 8, 3, 3, 3, 8, 0, 1, 1, 1, 9, 6, 4, 1, 6, 5, 8, 1, 0, 1, 0, 3, 0]
```

Note that your function should ignore punctuations signs, or any other character that is not a letter.

```
def count_letters(text):
    # Your code goes here:
```

```
    return
```

Another important feature that one can look at in frequency analysis are which groups of two letters (bigram) occur more often. For example, in English the most common bigrams are 'th' and 'he', both present in the word 'the'. If someone sees the following encrypted message:

JAD SUJ WY OLJ OF JAD GUE

A good guess would be that the 'ja' and 'ad' correspond to 'th' and 'eh'. Making possible to partially decrypt the message to:

the SUt WY OLt OF the GUE

Of course, in very small texts like this one it's impossible to go much further using frequency analysis and decipher the original message ("the cat is out of the bag"). We would need more text to be able to link each letter in the cipher to its corresponding letter.

Write a function `count_bigrams` that receive a text and returns two lists: one with all the bigrams present in the text, and another list with their correspondent count.

Ex:

```
bigrams, counts = count_bigrams("here and there")
print(bigrams)
print(counts)
```

```
['he', 'er', 're', 'an', 'nd', 'th']
```

```
[2, 2, 2, 1, 1, 1]
```

```
def count_bigrams(text):
    bigrams = []
    counts = []
    # Your code goes here:
```

```
    return bigrams, counts
```

Finally, implement the helper function *most_common* that receives a list of bigrams or letters, their counts and an integer *n*. The function then returns the *n* most common bigrams (letters).

Ex:

```
print(most_common(bigrams, counts, 3))
```

```
['he', 'er', 're']
```

```
def most_common(bigrams, counts, n):
    # Your code goes here:
```

```
    return
```

5.c. *Breaking the substitution cipher*

Now that you know more about frequency analysis and the substitution cipher, we can try to break it.

Some useful information is the frequency of each letter in the English alphabet and the frequency of the most common bigrams.

You can find this information [here](#) and [here](#).

With that in mind, we can analyze the following ciphertext:

KUM ZHF TK GKYSM HU MUCEFNTMA VMGGHRM, BQ ZM IUKZ BTG
 YHURXHRM, BG TK QBUA H ABQQMEMUT NYHBUTMPT KQ TJM GHVM
 YHURXHRM YKUR MUKXRJ TK QBY Y KUM GJMMT KE GK, HUA TJMU ZM
 CKXUT TJM KCCXEEMUCMG KQ MHCJ YMTTME. ZM CHYY TJM VKGT
 QEMDXMUTYF KCCXEEBUR YMTTME TJM 'QBEGT', TJM UMPT VKGT
 KCCXEEBUR YMTTME TJM 'GMCKUA' TJM QKYYKZBUR VKGT
 KCCXEEBUR YMTTME TJM 'TJBEA', HUA GK KU, XUTBY ZM HCCCKXUT
 QKE HYY TJM ABQQMEMUT YMTTMEG BU TJM NYHBUTMPT GHVNYM.
 TJMU ZM YKKI HT TJM CBNJME TMPT ZM ZHUT TK GKYSM HUA ZM
 HYGK CYHGGBQF BTG GFVWKYG. ZM QBUA TJM VKGT KCCXEEBUR
 GFVWKY HUA CJHURM BT TK TJM QKEV KQ TJM 'QBEGT' YMTTME KQ
 TJM NYHBUTMPT GHVNYM, TJM UMPT VKGT CKVVKU GFVWKY BG
 CJHURMA TK TJM QKEV KQ TJM 'GMCKUA' YMTTME, HUA TJM
 QKYYKZBUR VKGT CKVVKU GFVWKY BG CJHURMA TK TJM QKEV KQ TJM
 'TJBEA' YMTTME, HUA GK KU, XUTBY ZM HCCCKXUT QKE HYY GFVWKYG
 KQ TJM CEFNTKREHV ZM ZHUT TK GKYSM.

If we look for the 3 most common bigrams, we can find: ['tj', 'jm', 'hv'] Suggesting that 'tjm' is the word 'the'

We can try decode part of the text with that using our decoder function. Our partial key would be 'TJM' while our alphabet would be 'the'. That would give us the following text:

KUe ZHF tK GKYS eHU eUCEFNteA VeGGHRe, BQ Ze IUKZ BtG YHURXHRe,
 BG tK QBUA H ABQQeEeUt NYHBUTePt KQ the GHVe YHURXHRe YKUR
 eUKXRh tK QBY Y KUe Gheet KE GK, HUA theU Ze CKXUt the KCCXEEeUCeG
 KQ eHCh YetteE. Ze CHYY the VKGt QeEDXeUtYF KCCXEEBUR YetteE the
 'QBEGt', the UePt VKGt KCCXEEBUR YetteE the 'GeCKUA' the QKYYKZBUR
 VKGt KCCXEEBUR YetteE the 'thBEA', HUA GK KU, XUtBY Ze HCCCKXUt QKE
 HYY the ABQQeEeUt YetteEG BU the NYHBUTePt GHVNYe. theU Ze YKKI Ht
 the CBNheE tePt Ze ZHUt tK GKYS eHUA Ze HYGK CYHGGBQF BtG
 GFVWKYG. Ze QBUA the VKGt KCCXEEBUR GFVWKY HUA ChHUR eBt tK
 the QKEV KQ the 'QBEGt' YetteE KQ the NYHBUTePt GHVNYe, the UePt VKGt
 CKVVKU GFVWKY BG ChHUR eA tK the QKEV KQ the 'GeCKUA' YetteE, HUA
 the QKYYKZBUR VKGt CKVVKU GFVWKY BG ChHUR eA tK the QKEV KQ the
 'thBEA' YetteE, HUA GK KU, XUtBY Ze HCCCKXUt QKE HYY GFVWKYG KQ
 the CEFNtKREHV Ze ZHUt tK GKYS e.

In addition to that, there is also the word 'H' and 'tK' that appears very often. The most common one-letter words in English are 'I' and 'a'. In that case, we can guess that 'H' is 'a', since is often in the middle of the sentence. And 'tK' might indicate that 'K' is 'o'.

oUe ZaF to GoYSe aU eUCEFNteA VeGGaRe, BQ Ze IUoZ BtG YaURXaRe, BG to QBUA a ABQQeEeUt NYaBUtePt oQ the GaVe YaURXaRe YoUR eUoXRh to QBYY oUe Gheet oE Go, aUA theU Ze CoXUt the oCCXEEeUCeG oQ eaCh YetteE. Ze CaYY the VoGt QEDXeUtYF oCCXEEBUR YetteE the 'QBEGt', the UePt VoGt oCCXEEBUR YetteE the 'GeCoUA' the QoYYoZBUR VoGt oCCXEEBUR YetteE the 'thBEA', aUA Go oU, XUtBY Ze aCCoXUt QoE aYY the ABQQeEeUt YetteEG BU the NYaBUtePt GaVNYe. theU Ze YooI at the CBNheE tePt Ze ZaUt to GoYSe aUA Ze aYGo CYaGGBQF BtG GFVWoYG. Ze QBUA the VoGt oCCXEEBUR GFVWoY aUA ChaURe Bt to the QoEV oQ the 'QBEGt' YetteE oQ the NYaBUtePt GaVNYe, the UePt VoGt CoVVoU GFVWoY BG ChaUReA to the QoEV oQ the 'GeCoUA' YetteE, aUA the QoYYoZBUR VoGt CoVVoU GFVWoY BG ChaUReA to the QoEV oQ the 'thBEA' YetteE, aUA Go oU, XUtBY Ze aCCoXUt QoE aYY GFVWoYG oQ the CEFNtoREaV Ze ZaUt to GoYSe.

If we now look for the text we can find various words that might help us decode the key used. For example, the word 'YetteE' might map to 'letter' or 'better', the word 'Ze' to 'we' or 'me', and the word 'áU' to 'an'. You can also information on the most frequent letters to make your guesses.

If you keep going, you would find the key: "hwcamqrbliyvukndetgxszpfo" (using with the ordered alphabet)

And be able to decode the message to:

one way to solve an encrypted message, if we know its language, is to find a different plaintext of the same language long enough to fill one sheet or so, and then we count the occurrences of each letter. we call the most frequently occurring letter the 'first', the next most occurring letter the 'second' the following most occurring letter the 'third', and so on, until we account for all the different letters in the plaintext sample. then we look at the cipher text we want to solve and we also classify its symbols. we find the most occurring symbol and change it to the form of the 'first' letter of the plaintext sample, the next most common symbol is changed to the form of the 'second' letter, and the following most common symbol is changed to the form of the 'third' letter, and so on, until we account for all symbols of the cryptogram we want to solve.

Quote from "A Manuscript on Deciphering Cryptographic Messages", by Al-Kindi, from around 850AD

5.d. Challenge 1

Use the method described above to break the encryption of message in the file [challenge1.txt](#).

Note that your code does not need to be 100% automatic, and you don't need to decode the whole message all at once. You can have break part of the key and use that to get more information from the message (for example, decrypting the parts

that you know and trying to identify new letters. You can use the `substitution_cipher` function we implemented above to do that).

6. MATH PUZZLES

This chapter will help you get consolidate your knowledge of linear algebra. To solve the exercises in this section you will need to have some basic understanding of:

- numpy arrays
- loops and if statements
- recursion
- functions

This quest was inspired by Singh (1997).

6.a. *Problem:*

A Chessboard consists of 64 squares, organized in a 8 by 8 grid. The squares have alternating colors (usually black and white), in such a way that a square never share edges with a square of the same color.

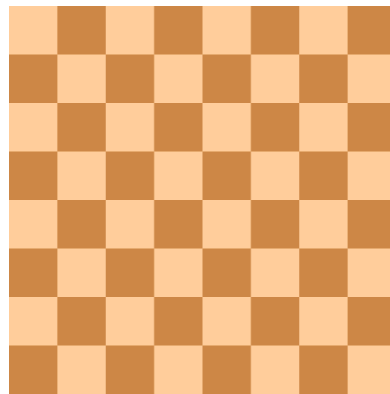


Figure 1: A typical chessboard. Source: Wikipedia contributors (2026)

Imagine that you have a modified Chessboard, in which the first and last squares of one of the diagonals were removed. This new board has now 62 squares. Is it possible to tile a this board with a set of 31 dominoes, so that each domino covers exactly two squares of the board and there are no squares uncovered?

Use your programming skills to find a viable tiling pattern for the board, or to prove that a pattern is not possible.

REFERENCES

- Singh, S. (1997). *Fermat's last theorem*. NY Books.
- Strang, G. (2009). *Introduction to Linear Algebra* (Fourth). Wellesley-Cambridge Press.
- Wikipedia contributors. (2026,). *Chessboard* — *Wikipedia, The Free Encyclopedia*. <https://en.wikipedia.org/w/index.php?title=Chessboard&oldid=1364131481>